WAVER

Product Scope v1.1

WAVER • Product Scope v1.0 • Baseline para revisão

PRODUCT SCOPE / ARCHITECTURE BASELINE

WAVER

AI Agent Platform — Product Scope v1.0

Documento consolidado para revisão de produto e arquitetura

Princípio do produto

Simple surface, powerful underneath. A plataforma abstrai a complexidade de LLMs e transforma agentes em unidades configuráveis, integráveis, governáveis e executáveis em produção.

Status: BASELINE PARA REVISÃO • Outubro de 2026

1. Executive Summary

WAVER é uma plataforma para criação, integração, execução e governança de agentes de IA voltados a atendimento e tarefas empresariais. O produto não pretende substituir todos os sistemas de uma empresa; ele funciona como uma camada inteligente sobre sistemas existentes e, quando necessário, pode oferecer aplicações nativas.

A arquitetura é multi-tenant e organizada por domínios de negócio. Dentro de um Workspace, áreas como Comercial, Suporte e Marketing são tratadas como fronteiras independentes, com agentes, conhecimento, dados e políticas próprios.

Decisão central

Na criação de um agente, o usuário escolhe uma estratégia comportamental exclusiva: Objective-driven ou Flow-

driven. Essa escolha determina a experiência de configuração e o motor de execução do agente.

2. Product Vision

 Criar agentes de IA sem exigir que o usuário final escreva prompts técnicos.

 Conectar inteligência aos sistemas que a empresa já utiliza.

 Permitir agentes de atendimento e agentes orientados a tarefas sobre a mesma plataforma.

 Oferecer ferramentas, integrações, conhecimento, canais, runtime e governança como capacidades reutilizáveis.

 Ser distribuível por revendas sem necessariamente ocultar a marca da plataforma.

 Escalar de configurações simples para cenários empresariais complexos sem transformar a interface em um painel de engenharia.

WAVER • Product Scope v1.0 • Baseline para revisão

3. Product Principles

Princípio Aplicação

Abstraction over prompts O usuário configura intenção, objetivos, fluxos e políticas; prompts internos são derivados/compilados pela

plataforma.

Composable capabilities Agents consomem Tools; Tools podem vir de integrações, APIs customizadas ou apps nativos.

Domain isolation Sales, Support, Marketing etc. possuem fronteiras próprias de dados e configuração.

Version everything important Agent definitions, tools, policies, flows e deployments devem ter histórico e rollback quando aplicável.

Builder ≠@ Runtime O Builder define; o Runtime executa versões publicadas.

Progressive disclosure Configurações avançadas existem, mas aparecem somente quando necessárias.

Governed autonomy Autonomia do agente é limitada por permissions, policies, approvals, quotas e audit trail.

4. Organizational Model & Multi-Tenancy

Estrutura conceitual principal:

Organization Workspace Domain Agent AgentVersion → → → → →

Deployment

4.1 Organization

 Entidade comercial/tenant de maior nível.

 Relaciona plano, assinatura, limites, usuários e workspaces.

4.2 Workspace

 Ambiente organizacional onde usuários e recursos são agrupados.

 Pode conter múltiplos Domains.

4.3 Domain

 Bounded context de negócio: Comercial, Suporte, Marketing, Financeiro ou domínio customizado.

 Possui seus próprios Agents, Knowledge Bases, políticas, configurações e escopo de dados.

 Não deve ser confundido com tipo fixo de agente; é uma fronteira organizacional.

4.4 Identity & Access

 User, Membership, Role, Permission, Service Account e API Key.

 RBAC como base; ABAC/policies contextuais pode ser adicionado para cenários avançados.

 Permissões devem respeitar Organization, Workspace, Domain e recurso.

5. Platform Domains

Domínio da plataforma Responsabilidade

Agent Builder Definição, configuração, versionamento, teste e publicação dos agentes.

Tools & Capabilities Catálogo de ações reutilizáveis, schemas, versões, permissões e políticas de execução.

Integration Hub Connectors, autenticação, credenciais, webhooks e

conexões externas.

Knowledge Management Fontes, documentos, ingestão, indexação, retrieval e escopo de conhecimento.

Agent Runtime Execução de agentes, sessões, estado, tool calls, retries, filas e concorrência.

Triggers & Workflows Eventos, schedules, webhooks e orquestrações.

Channels Web Chat, WhatsApp, E-mail, API, Voice e canais futuros.

Testing & Evaluation Test suites, cenários, regressão, avaliações e métricas.

Deployment & Environments Draft, staging, production, releases e rollback.

Observability & Governance Logs, traces, custos, auditoria, quotas, rate limits e políticas.

Billing & Licensing Planos, subscriptions, usage metering, licenças e relacionamento com revendas.

Native Apps CRM, inbox/support, agenda e outros apps opcionais construídos sobre a plataforma.

6. Agent Builder

O Agent Builder é a unidade de autoria. Ele não executa diretamente o estado editável. Seu resultado é uma AgentVersion versionada, que pode ser testada e posteriormente publicada em um Environment.

6.1 Agent Definition

 Identity: nome, descrição, domínio, finalidade e metadados.

 AgentStrategy: OBJECTIVE_DRIVEN ou FLOW_DRIVEN.

 Model configuration: provider, model, fallback/routing e configurações avançadas.

 Knowledge bindings.

 Tool bindings e permissões.

 Policies e guardrails.

 Memory configuration.

 Channels e triggers.

 Evaluation/test configuration.

6.2 Prompt Abstraction

O produto não deve depender de uma grande caixa de “System Prompt” como experiência principal. A plataforma compõe internamente instruções técnicas a partir de estruturas de produto: identidade, estratégia, objetivos/fluxos, políticas, conhecimento, ferramentas, contexto e preferências de comunicação.

Configuração avançada

Parâmetros como temperature, top_p, token limits, routing e fallback continuam disponíveis para usuários avançados, mas não devem dominar a experiência padrão.

7. Agent Strategies

7.1 Objective-driven Agent

O agente recebe resultados que precisa alcançar, mas possui liberdade para conduzir a conversa de forma natural.

Elemento Descrição

Objective Resultado a alcançar, como obter CNPJ, cidade ou motivação.

Priority Importância relativa do objetivo.

Completion Condition Critério estruturado para considerar o objetivo concluído.

Dependencies Objetivos que precisam ser satisfeitos antes de outro.

Attempts / Retry Policy Controle de novas tentativas sem tornar a conversa repetitiva.

State PENDING, IN_PROGRESS, COMPLETED, SKIPPED ou FAILED.

Extracted Value Valor obtido e evidência/contexto que levou à conclusão.

Comportamento esperado: se o usuário fizer uma pergunta enquanto existe um objetivo pendente, o agente pode responder primeiro e depois retomar naturalmente o objetivo. O runtime mantém ObjectiveState separado da mensagem atual.

7.2 Flow-driven Agent

O agente opera por etapas e transições explícitas. É indicado para processos que exigem sequência, compliance ou decisões determinísticas.

Elemento Descrição

Flow Definição versionada do processo.

FlowNode Etapa: Message, Question, Action, Condition, Delay, Transfer, Objective Group, End etc.

FlowTransition Ligação entre nodes com condição de transição.

FlowState Posição atual da execução.

Variables Dados utilizados pelo fluxo.

Actions Tool calls ou operações associadas às etapas.

7.3 Shared Capabilities

 Knowledge, Tools, Policies, Memory, Channels, Model, Testing e Observability são comuns aos dois modos.

 A UI de Objectives e a UI de Flow Builder são separadas.

 A estratégia é escolhida na criação do agente e tratada como propriedade estrutural/versionada.

8. Policies, Guardrails & Autonomy

Policies são regras invariáveis ou condicionais que limitam o comportamento. Exemplos: não inventar informação, não conceder desconto sem aprovação, não solicitar novamente um dado conhecido, encaminhar intenção de cancelamento para suporte.

 Policy scope: Organization, Workspace, Domain, Agent ou Tool.

 Severity/Priority e comportamento em caso de violação.

 Human approval para ações sensíveis.

 Execution limits, rate limits e spending limits.

 Auditabilidade de decisões e ações.

 PII/data handling e retention policies.

9. Tools & Capability Platform

Tool é uma capacidade executável independente do Agent. O Agent referencia uma Tool por binding e recebe somente as permissões necessárias.

Entidade Responsabilidade

Tool Identidade semântica da capacidade.

ToolVersion Contrato versionado com input/output schema.

ToolBinding Vínculo de uma AgentVersion com a Tool.

ToolPermission Ações/escopos autorizados.

ExecutionPolicy Timeout, retries, idempotency, approval, limits.

ToolExecution Registro de cada execução, resultado, erro e duração.

 Uma Tool pode ser Native, Integration-backed, API ou Custom.

 Exemplos: create_lead, find_customer, update_deal, send_message, schedule_meeting.

 O agente não precisa conhecer o produto de origem; ele consome a capability.

10. Integration Hub

 Connector: integração suportada pela plataforma.

 Connection: instalação/configuração do connector para um tenant/domain.

 Credential: segredo ou autorização; deve ser isolado e protegido.

 OAuth/API Key/Basic/Custom Auth.

 Inbound webhooks e outbound API calls.

 Custom REST/API connector para sistemas não suportados nativamente.

 Health status, reconnect, scopes e credential rotation.

11. Knowledge & Memory

11.1 Knowledge

 KnowledgeBase por Domain/Agent conforme escopo.

 KnowledgeSource: documentos, websites, databases, APIs e conectores.

 Ingestion pipeline, chunking, metadata, indexing e retrieval.

 Permissões e filtros de retrieval devem respeitar tenant/domain.

11.2 Memory

 Conversation/session memory.

 Long-term user/customer memory quando permitido.

 Business/domain memory.

 Retention, expiration, consent e exclusão.

 Memória não deve ser confundida com Knowledge Base.

12. Agent Runtime

Runtime executa versões publicadas. Ele recebe eventos/canais, carrega contexto, aplica policies, escolhe o mecanismo de estratégia, executa tools e persiste estado.

1. Resolver tenant, workspace, domain, agent deployment e versão.

2. Criar/retomar Session e Conversation.

3. Carregar ExecutionContext, Memory, Knowledge bindings e Policies.

4. Executar Objective Engine ou Flow Engine conforme AgentStrategy.

5. Realizar model calls e tool calls autorizadas.

6. Persistir states, messages, traces, costs e audit events.

7. Responder pelo Channel correspondente.

 Retries e idempotência para operações seguras.

 Queueing e concurrency control.

 Timeout/cancellation.

 Fallback de modelo/provedor.

 Human handoff.

 Resumable executions para processos longos.

13. Channels

Canal Escopo esperado

Web Chat Widget/SDK e inbox associado.

WhatsApp Mensagens, templates e eventos do provedor.

E-mail Inbound/outbound e threading.

API Execução programática e eventos.

Voice Canal futuro para conversação por voz.

Custom Channel Adapter para canais específicos.

WAVER • Product Scope v1.0 • Baseline para revisão

Channel deve ser adapter; lógica de negócio permanece no Runtime/Agent.

14. Triggers & Automation

 Message received.

 Webhook.

 Schedule.

 System/business event.

 API invocation.

 Manual execution.

Triggers iniciam execuções. Workflows coordenam sequências mais amplas de ações e podem invocar Agents/Tools.

15. Testing & Evaluation

 Playground isolado por versão.

 Test Suite e Test Case.

 Conversation scenarios e datasets.

 Expected outcomes, tool-call assertions e policy assertions.

 Automated evaluation e human review.

 Regression testing antes de deployment.

 Comparação entre AgentVersions/modelos.

 Métricas: task completion, objective completion, handoff rate, latency, cost, tool errors e quality scores.

16. Deployment & Environments

 Draft: configuração editável.

 Version: snapshot versionado da definição.

 Environment: development/staging/production ou custom.

 Deployment: vínculo entre AgentVersion e Environment.

 Release/Publish e rollback.

 Configuração e secrets por ambiente.

 Possibilidade de canary/gradual rollout no futuro.

Regra arquitetural

Editar o Agent não altera automaticamente o que está em produção. O Runtime executa uma versão publicada por Deployment.

17. Observability & Governance

 Structured logs.

 Distributed traces por AgentRun/ToolExecution.

 Token/model usage e custos.

 Latency e error metrics.

 Audit events para mudanças administrativas e ações sensíveis.

 Quotas, rate limits e budget controls.

 Dashboards por Organization, Workspace, Domain, Agent e versão.

 Alerting e incident diagnostics.

 Data retention e export.

18. Billing, Licensing & Reseller Model

Direção atual: revendas funcionam como canal comercial da plataforma, sem obrigatoriedade de white-label. A marca do produto permanece visível por padrão.

 Plan e Subscription.

 UsageRecord para metering.

 Licenses/seats/agent limits conforme modelo comercial.

 Reseller account, customer relationship, commission/margin e license allocation.

 Entitlements por plano.

 Overage/limits e billing events.

Ainda em aberto

O modelo financeiro exato de revendas, cobrança, comissionamento, ownership do cliente e possíveis níveis de branding ainda precisa ser definido.

19. Native Apps

Apps nativos não são o núcleo da plataforma. Eles são produtos construídos sobre as mesmas capacidades e contratos.

 Native CRM básico.

 Support/Inbox.

 Calendar/Scheduling.

 Outros módulos futuros.

Um agente deve consumir a mesma semântica de Tool independentemente de a ação vir de um app nativo ou de uma integração externa.

20. User Experience Architecture

A interface deve ser clara, técnica e robusta sem expor detalhes de implementação desnecessários.

 Paleta clara: branco/creme com laranja, vermelho e amarelo; evitar superfícies escuras como linguagem principal.

 Controles finos e compactos; evitar botões excessivamente grandes.

 Hierarquia visual densa, mas organizada.

 Progressive disclosure para configurações avançadas.

 Telas separadas por responsabilidade.

 Creation wizard define Domain, identidade, AgentStrategy e capacidades comuns.

 Objective Builder e Flow Builder possuem experiências próprias.

 Testing/preview disponível sem misturar todas as configurações na mesma tela.

 Parâmetros técnicos de LLM aparecem em Advanced Settings; experiência padrão usa abstrações compreensíveis.

21. Conceptual Core Entities

Área Entidades principais

Tenancy Organization, Workspace, Domain, User, Membership,
Role, Permission
Agent Agent, AgentVersion, AgentStrategy, ModelConfig,
BehaviorConfig
Objectives ObjectiveDefinition, ObjectiveState, CompletionCondition,
ExtractedValue
Flows Flow, FlowVersion, FlowNode, FlowTransition, FlowState,
FlowVariable
Policies Policy, PolicyBinding, ApprovalPolicy, GuardrailPolicy
Knowledge KnowledgeBase, KnowledgeSource, Document, Index,
RetrievalConfig
Memory MemoryConfig, MemoryEntry, RetentionPolicy
Tools Tool, ToolVersion, ToolBinding, ToolPermission,
ToolExecution
Integrations Connector, Connection, Credential, WebhookEndpoint
Runtime AgentRun, ExecutionContext, Session, Conversation,
Message
Automation Trigger, Workflow, WorkflowStep, Schedule
Delivery Environment, Deployment, Release
Evaluation TestSuite, TestCase, EvaluationRun, EvaluationResult
Governance Log, Trace, Metric, AuditEvent, Quota, RateLimit
Channels Channel, ChannelConnection, ChannelSession
Commercial Plan, Subscription, Entitlement, UsageRecord, Reseller,
License

22. Critical Non-Functional Requirements

 Strict tenant isolation.
 Encryption in transit and at rest; secret management.
 Horizontal scalability of Runtime.
 High availability for production channels.
 Idempotency for side-effecting tool calls.
 Auditable authorization decisions.

 Version compatibility and migrations.
 Data deletion/retention mechanisms.
 Provider abstraction to support multiple LLM vendors.
 Cost controls and observability from the beginning.
 Extensible connector/tool SDK.
 Event-driven architecture where asynchronous execution is beneficial.

23. Decisions Already Made

 The product is broader than an SDR agent; it is an agent platform.

 The Agent Builder is central, but not the whole platform.

 The platform integrates with existing business systems instead of trying to replace all of them.

 Native Apps are optional products built on platform capabilities.

 The system is organized by business Domains inside Workspaces.

 Agents belong to Domains and should not implicitly share all data across domains.

 Tools are independent reusable capabilities.

 Builder and Runtime are separate responsibilities.

 Production execution uses published/versioned definitions.

 Agent behavior is not primarily configured through raw prompts.

 AgentStrategy is Objective-driven or Flow-driven, selected during creation.

 Objectives are flexible conversational goals; Flows are deterministic process structures.

 Policies define invariant behavioral/operational boundaries.

 UI uses progressive disclosure and hides raw LLM complexity by default.

 Current visual direction is light, warm and technical: white/cream + orange/red/yellow.

24. Open Decisions / Review Checklist

Decisão aberta Pergunta para revisão

Domain data isolation Separate physical databases, schemas, row-level isolation,

or configurable strategy?

Strategy changes Can an existing agent migrate Objective Flow, or ↔

require a new agent/version?

Hybrid execution Do we ever allow an explicit Objective node inside a Flow

without turning the agent into a hybrid strategy?

Objective engine How are completion conditions represented: extraction

schema, rules, evaluators, or all three?

Human handoff Ownership, queueing, SLA, inbox and resume semantics.

Conversation ownership Can one conversation move between agents/domains?

Knowledge Vector store strategy, freshness, citations and permission-

aware retrieval.

Memory What may be remembered, for how long, and at what

scopes?

Tool security Approval levels, sandboxing, egress restrictions and

credential boundaries.

Integrations Connector SDK and custom connector deployment model.

Channels Which channel is first-class at launch and how provider

abstraction works.

Runtime Queue/event technology, state persistence and long-

running execution model.

Evaluation Quality metrics and release gates required before

production.

Multi-region/data residency Whether regional isolation will be required.

Billing Usage unit: tokens, runs, conversations, agents, tool calls, seats, or hybrid.

Resellers Commercial hierarchy, licensing, commission, customer ownership and branding.

Native Apps Which apps are strategic versus merely examples/reference implementations.

Compliance LGPD controls, audit retention, consent, DPA and enterprise requirements.

Model routing Rules for multi-provider fallback, cost optimization and customer-provided keys.

Marketplace Whether Tools/Connectors become publishable third-party extensions.

25. Proposed Review Order

8. Validate product boundaries and terminology.

9. Validate Organization Workspace Domain hierarchy.→ →

10. Finalize Agent lifecycle and AgentVersion model.

11. Specify Objective-driven execution in detail.

12. Specify Flow-driven execution in detail.

13. Finalize Tool contract and permissions.

14. Finalize Integration Hub and credential model.

15. Define Runtime state machine and event model.

16. Define Knowledge/Memory isolation.

17. Define deployment, evaluation and observability requirements.

18. Define reseller/licensing and billing model.

19. Only then freeze UI information architecture and implementation roadmap.

26. Scope Boundary

Este documento descreve a arquitetura-alvo e o escopo funcional atualmente concebido. Ele não define ainda tecnologias específicas de infraestrutura, banco de dados, filas, vector database, framework frontend/backend ou provedor de cloud. Essas escolhas devem ser derivadas após a validação dos requisitos e dos fluxos críticos.

Próximo artefato recomendado

Depois da revisão deste escopo: Master Domain Model + UMLs específicas por domínio, começando por Agent Builder / Agent Runtime e pelos state machines de Objective-driven e Flow-driven.